# EXPERIMENT 2

Sanket Patil D20A-25

## Aim

**To create and implement a basic Blockchain using Python with SHA-256 hashing, Proof-of-Work, transaction handling, blockchain validation, and Flask APIs.**

# Theory

A **blockchain** is a distributed digital ledger in which data is stored in a sequence of interconnected blocks. Each block contains information such as an index, timestamp, transactions, proof value, and the hash of the previous block. The previous hash connects one block to another and helps maintain the integrity of the blockchain.

## Components of Blockchain

**1. Block:**
A block stores blockchain-related information such as transactions, timestamp, proof, index and previous hash.

**2. Genesis Block:**
The Genesis Block is the first block of the blockchain. It is created when the blockchain is initialized.

**3. Transaction:**
A transaction represents the transfer of information or value between a sender and receiver.

**4. Proof-of-Work:**
Proof-of-Work is a computational process used to find a valid proof satisfying a predefined condition. In this experiment, the SHA-256 hash generated during mining must begin with three zeros (000).

**5. SHA-256:**
SHA-256 is a cryptographic hashing algorithm used to generate a hash for blockchain data.

**6. Previous Hash:**
The previous hash stores the hash of the previous block. It connects consecutive blocks and helps detect modification of blockchain data.

**7. Blockchain Validation:**
Blockchain validation checks whether the previous hashes and Proof-of-Work of the blocks are correct.

**8. Flask API:**
Flask is used to create HTTP APIs through which blockchain operations can be performed.

**9. Postman:**
Postman is used to send requests to the Flask APIs and examine their responses.

# Proof-of-Work

In this experiment, Proof-of-Work is implemented by repeatedly changing the proof value and calculating:

SHA-256(new_proof² - previous_proof²)

The proof is accepted only when the resulting hash begins with:

000

Thus, the computer performs repeated calculations until a valid proof is found.

# Procedure

## Step 1: Install and Import Required Libraries

Install Flask and import the required Python libraries.

```
!pip install flask==2.2.5

import datetime
import hashlib
import json

from flask import Flask, jsonif
```


## Step 2: Implement the Blockchain

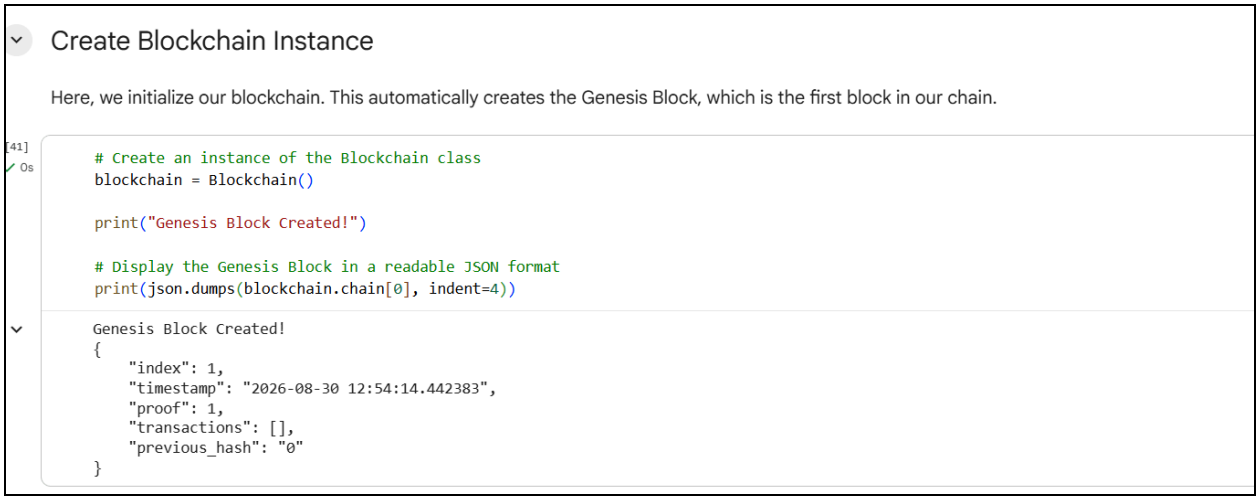
Create a Blockchain class to manage blocks, transactions, Proof-of-Work, hashing and validation.

```
class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []

        # Create Genesis Block
        self.create_block(
            proof=1,
            previous_hash='0'
        )

    def create_block(self, proof, previous_hash):

        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'transactions': self.transactions.copy(),
            'previous_hash': previous_hash
        }

        self.chain.append(block)
```

```
    return block

def get_previous_block(self):
    return self.chain[-1]

def create_Transactions(self, sender, receiver, amount):

    transaction = {
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    }

    self.transactions.append(transaction)

    return self.get_previous_block()['index'] + 1

def proof_of_work(self, previous_proof):

    new_proof = 1

    while True:

        hash_operation = hashlib.sha256(
            str(
                new_proof ** 2 - previous_proof ** 2
            ).encode()
        ).hexdigest()

        if hash_operation[:3] == '000':
            return new_proof

        new_proof += 1

def hash(self, block):

    encoded_block = json.dumps(
        block,
        sort_keys=True
```

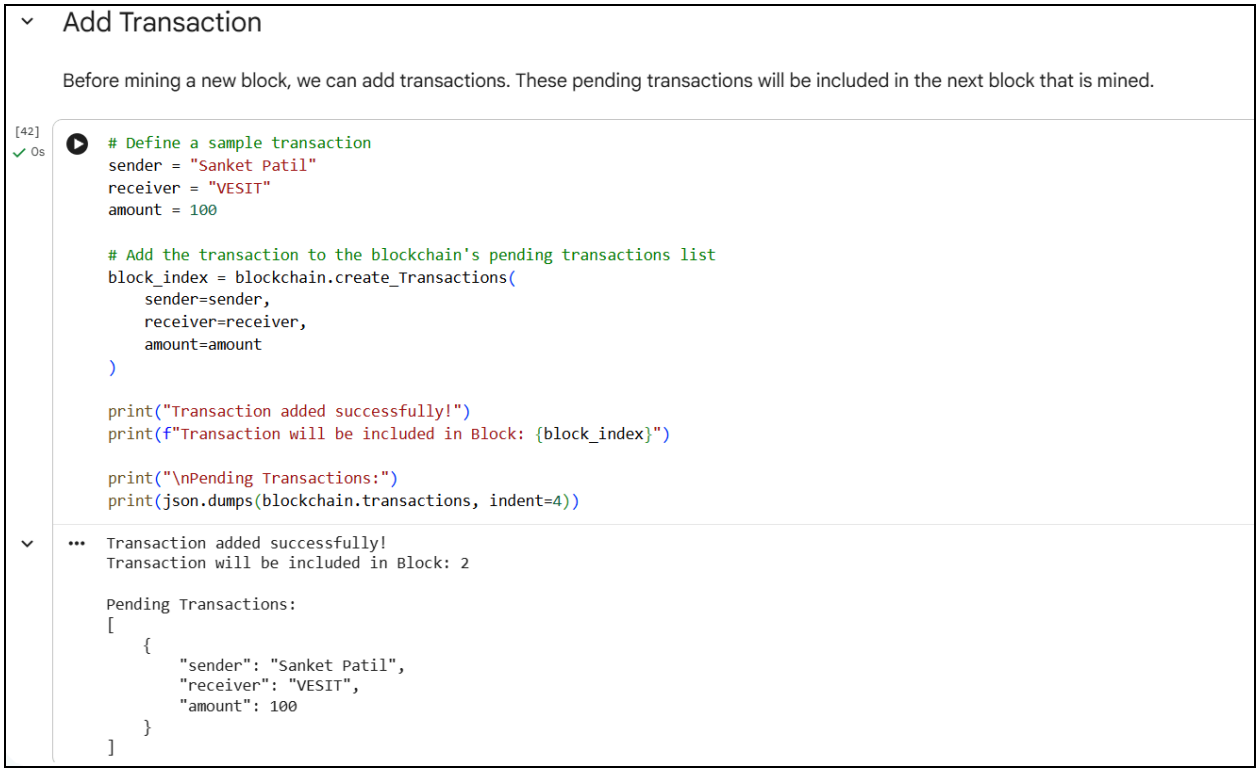
```
        ).encode()

        return hashlib.sha256(
            encoded_block
        ).hexdigest()

    def is_chain_valid(self, chain):

        previous_block = chain[0]
        block_index = 1

        while block_index < len(chain):

            block = chain[block_index]

            if block['previous_hash'] != self.hash(previous_block):
                return False

            previous_proof = previous_block['proof']
            proof = block['proof']

            hash_operation = hashlib.sha256(
                str(
                    proof ** 2 - previous_proof ** 2
                ).encode()
            ).hexdigest()

            if hash_operation[:3] != '000':
                return False

            previous_block = block
            block_index += 1

        return True
```

The class follows the structure of the supplied experiment, including creation of the Genesis Block, transactions, PoW, SHA-256 hashing and validation.




## Step 3: Create Blockchain and Add Transaction

```
blockchain = Blockchain()

block_index = blockchain.create_Transactions(
    sender="Sanket Patil",
    receiver="VESIT",
    amount=100
)

print("Transaction added successfully!")
print("Transaction will be included in Block:", block_index)

print("\nPending Transactions:")
print(blockchain.transactions)
```




## Step 4: Mine the Block

```
previous_block = blockchain.get_previous_block()

previous_proof = previous_block['proof']

print("Mining block...")

proof = blockchain.proof_of_work(previous_proof)

print("Proof found:", proof)

previous_hash = blockchain.hash(previous_block)

new_block = blockchain.create_block(
    proof,
    previous_hash
)

blockchain.transactions.clear()

print("\nBlock mined successfully!")
```

print(new_block)

The proof value is generated dynamically, so the number obtained during execution may differ from the reference output. The reference experiment, for example, obtained 533.

```
Mining block...
Proof found: 533
Block mined successfully!
{
    "index": 2,
    "timestamp": "2026-08-30 12:54:14.458012",
    "proof": 533,
    "transactions": [
        {
            "sender": "Sanket Patil",
            "receiver": "VESIT",
            "amount": 100
        }
    ],
    "previous_hash": "ee7a60d5b6a4404db3a61691c003b9853e743ec2a40b2e8197b6e5cb95d1b1a6"
}
```

**Step 5: Display and Validate Blockchain**

```
print("================================")
print("COMPLETE BLOCKCHAIN")
print("================================")

print(json.dumps(blockchain.chain, indent=4))

print("\n================================")
print("BLOCKCHAIN VALIDITY")
print("================================")

if blockchain.is_chain_valid(blockchain.chain):
    print("All good. The Blockchain is valid.")
else:
    print("The Blockchain is not valid.")
```

Expected validity result:

All good. The Blockchain is valid.

The supplied experiment demonstrates the same successful validation.

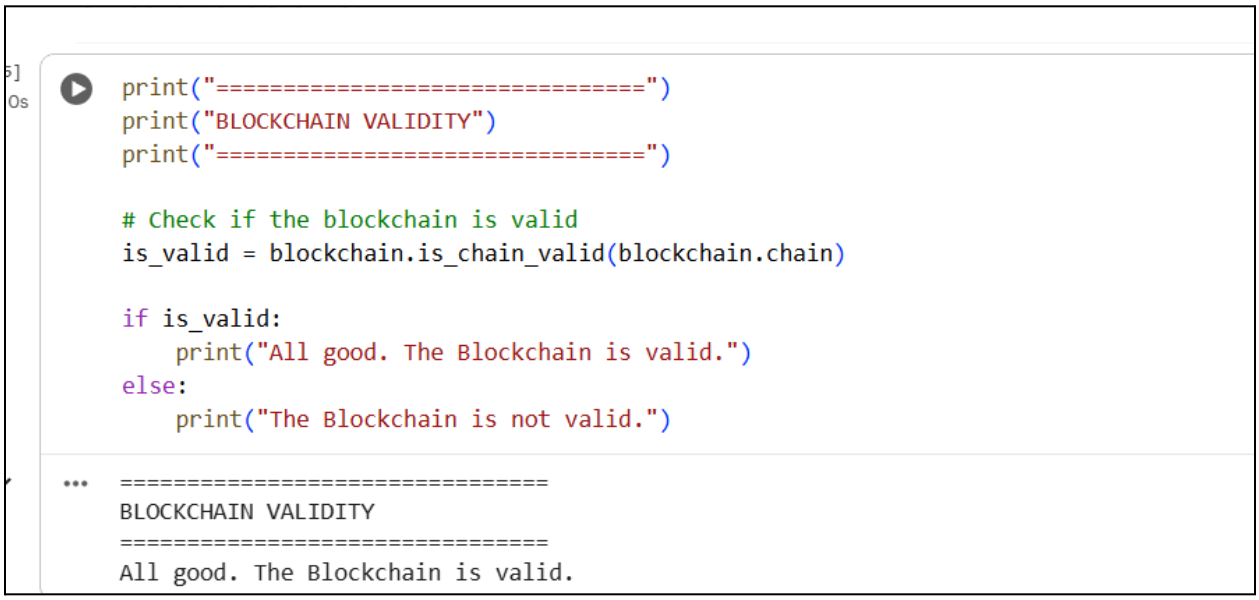
```
================================
COMPLETE BLOCKCHAIN
================================
[
    {
        "index": 1,
        "timestamp": "2026-08-30 12:54:14.442383",
        "proof": 1,
        "transactions": [],
        "previous_hash": "0"
    },
    {
        "index": 2,
        "timestamp": "2026-08-30 12:54:14.458012",
        "proof": 533,
        "transactions": [
            {
                "sender": "Sanket Patil",
                "receiver": "VESIT",
                "amount": 100
            }
        ],
        "previous_hash": "ee7a60d5b6a4404db3a61691c003b9853e743ec2a40b2e8197b6e5cb95d1b1a6"
    }
]
```


# Step 6: Implement Flask APIs

```
app = Flask(__name__)
@app.route('/mine_block', methods=['GET'])
def mine_block():

    if len(blockchain.transactions) == 0:
        return jsonify({
            'message': 'No transactions available.'
        }), 400

    previous_block = blockchain.get_previous_block()

    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)

    previous_hash = blockchain.hash(previous_block)

    block = blockchain.create_block(
        proof,
        previous_hash
    )

    blockchain.transactions.clear()
```

```
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'transactions': block['transactions'],
        'previous_hash': block['previous_hash']
    }

    return jsonify(response), 200


@app.route('/get_chain', methods=['GET'])
def get_chain():

    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }

    return jsonify(response), 200


@app.route('/is_valid', methods=['GET'])
def is_valid():

    valid = blockchain.is_chain_valid(
        blockchain.chain
    )

    if valid:
        response = {
            'message': 'All good. The Blockchain is valid.'
        }
    else:
        response = {
            'message': 'The Blockchain is not valid.'
        }

    return jsonify(response), 200
```

The three APIs implemented are /mine_block, /get_chain and /is_valid



```
print("================================")
print("BLOCKCHAIN VALIDITY")
print("================================")

# Check if the blockchain is valid
is_valid = blockchain.is_chain_valid(blockchain.chain)

if is_valid:
    print("All good. The Blockchain is valid.")
else:
    print("The Blockchain is not valid.")
```

```
================================
BLOCKCHAIN VALIDITY
================================
All good. The Blockchain is valid.
```

# Step 7: Run Flask Using ngrok

!pip install pyngrok

Then:

from pyngrok import ngrok

ngrok.set_auth_token("YOUR_NGROK_AUTH_TOKEN")

Start Flask:

import threading

```
def run_flask():
    app.run(
        host='0.0.0.0',
        port=5000,
        debug=False,
        use_reloader=False
    )

thread = threading.Thread(target=run_flask)
thread.start()
```

Generate the public URL:

```
import time
from pyngrok import ngrok

time.sleep(3)

public_url = ngrok.connect(5000)

print("Flask URL:")
print(public_url)
```

The reference experiment uses pyngrok to expose the Colab Flask server through a public URL.

# Step 8: Test Using Postman

## API 1 — Mine Block

**Method:** GET

https://YOUR-NGROK-URL/mine_block

Expected response:

```
{
  "message": "Congratulations, you just mined a block!",
  "index": 3,
  "proof": 1234,
  "transactions": [...]
}
```

## API 2 — Check Validity

**Method:** GET

https://YOUR-NGROK-URL/is_valid

Expected:

```
{
  "message": "All good. The Blockchain is valid."
}
```

### API 3 — Get Blockchain

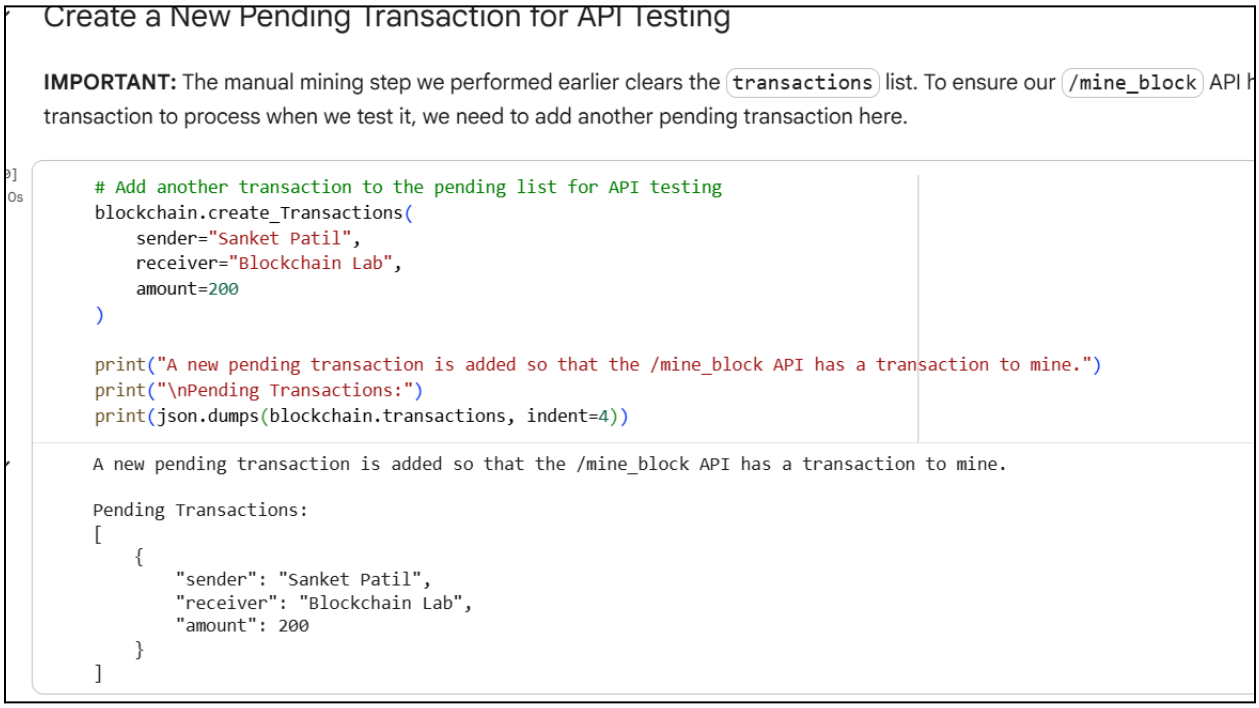
**Method:** GET

https://YOUR-NGROK-URL/get_chain

Expected response contains:

```
{
  "chain": [...],
  "length": 2
}
```

These are the three API tests demonstrated in the supplied experiment.



```python
# Give Flask a few seconds to start up completely
time.sleep(3)

# Connect ngrok to the Flask port (5000)
public_url = ngrok.connect(5000)

print("Flask URL:")
print(public_url)

print("\nAPI Endpoints:")
print("Mine Block:", str(public_url) + "/mine_block")
print("Get Chain:", str(public_url) + "/get_chain")
print("Check Validity:", str(public_url) + "/is_valid")

print("\nUse these URLs in Postman to test the APIs.")
```

```
Flask URL:
NgrokTunnel: "https://kisser-booted-very.ngrok-free.dev" -> "http://localhost:5000"

API Endpoints:
Mine Block: NgrokTunnel: "https://kisser-booted-very.ngrok-free.dev" -> "http://localhost:5000"/mine_block
Get Chain: NgrokTunnel: "https://kisser-booted-very.ngrok-free.dev" -> "http://localhost:5000"/get_chain
Check Validity: NgrokTunnel: "https://kisser-booted-very.ngrok-free.dev" -> "http://localhost:5000"/is_valid

Use these URLs in Postman to test the APIs.
```

# Conclusion

The basic blockchain was successfully created using Python. The experiment demonstrated the working of blocks, transactions, SHA-256 hashing, Proof-of-Work, previous-hash linking and blockchain validation. Flask APIs were also successfully implemented to mine blocks, retrieve the blockchain and verify its validity through Postman.